Prototype Implementation (Deliverable D2)

SBOM Supply Chain Risk Scoring: A Contextual Component Risk Ranking Framework

William A. Davis
CS 690 Graduate Research Project, Fall 2026
Advisor: Dr. Omar Ochoa
Covers Task F (parser and dependency graph), Task G (vulnerability enrichment) and Task H (scoring, ranking and baseline)
November 6, 2026

---

1. Summary

Deliverable D2 is the working pipeline: it reads a CycloneDX SBOM, rebuilds the dependency graph, looks up every component's known vulnerabilities, scores and ranks the vulnerable components with the model defined in the D1 Scoring Specification, explains each rank in one sentence, and produces the CVSS-only baseline ranking from the same data. Its success criterion is that the pipeline executes end to end on at least one test project without error. It does so on PyGoat, whose results are reported in Section 5, and the same commands run unchanged on the other three test projects.

The scoring follows D1 exactly. No factor, weight or formula was changed during implementation; the few points D1 left open are recorded in Section 7.

2. Pipeline

The pipeline is a Python 3 command-line tool named sbomrisk with no third-party runtime dependencies. Each stage is a separate module, so it can be tested and explained on its own.

Stage	Module	What it does
1. Parse	parser.py	Reads a CycloneDX 1.4 to 1.6 JSON SBOM [1] and builds one record per component with its package URL
2. Graph	graph.py	Rebuilds the dependency graph from the SBOM's dependencies section and measures each component's shortest depth from the application
3. Enrich	enrich.py	Finds each component's vulnerabilities in OSV [2], adds CVSS from NVD [3], EPSS [4], CISA KEV status [5] and the version's release date, and caches every response
4. Score and rank	score.py	Computes the five factors and the risk score, picks each component's driving vulnerability, orders the components, and writes the explanation
5. Baseline	score.py	Orders the same components by their highest CVSS score alone
6. Evaluate	evaluate.py	Compares the two rankings and runs the weight sensitivity analysis (used for D3)

The commands are run from PowerShell inside the project folder. Enrichment is the only stage that needs internet access; ranking and evaluation run entirely from the saved results.

```
sbomrisk enrich data\sboms\pygoat-v2.0.1.cdx.json
sbomrisk rank data\sboms\pygoat-v2.0.1.cdx.json
sbomrisk evaluate
```

3. Scoring

Each vulnerability v receives a vulnerability score from its severity S, its EPSS exploit probability E and its known-exploitation value X. A component c is scored on its single worst vulnerability, plus its dependency depth D and its version age A:

$$VulnScore(v) = w_1 S(v) + w_2 E(v) + w_3 X(v)$$

$$Risk(c) = \max_{v \in c} VulnScore(v) + w_4 D(c) + w_5 A(c)$$

The default weights are 0.30, 0.25, 0.20, 0.15 and 0.10. They are stored in the configuration file weights.json rather than in code, so the sensitivity runs and any future recalibration need no code change. The factor definitions, missing-data rules and tie-breaking rules are those in Sections 3 and 4 of D1.

4. Outputs

The rank command writes its results into the same dated folder as the enrichment it used, for example results\ pygoat-v2.0.1-2026-11-06.

File	Contents
ranking.json, ranking.csv	Every vulnerable component with its rank, baseline rank and movement, score, five factor values and their point contributions, driving vulnerability, full vulnerability list, depth and path, release date, data-quality flags and explanation
baseline.csv	The CVSS-only ranking of the same components
not-ranked.csv	Components with no known vulnerabilities, and any that could not be assessed, with the reason
run.json	Analysis date, tool version, weights, SBOM hash, data sources, where the API responses are cached, and counts of every data-quality flag
enrichment.json	The raw enrichment result the ranking was computed from

The CSV files open directly in Excel. Because every API response is cached and every run records its analysis date and SBOM hash, any result can be reproduced exactly.

5. End-to-End Run on PyGoat

PyGoat 2.0.1 was first enriched on October 23, 2026 (milestone M2). For this deliverable it was enriched again on November 6, 2026, the same analysis date as the other three test projects, and ranked from those results. The SBOM lists 33 components. OSV reports 145 vulnerabilities in 14 of them, so 14 components are ranked; 19 have no known vulnerabilities and none were unassessable. 136 of the 145 vulnerabilities have both a CVSS score from NVD and an EPSS probability, and one, CVE-2023-4863, is in the CISA KEV catalog.

Top 10

Rank	Component	Score	CVSS-only rank	Driving vulnerability (CVSS)	Largest factors
1	pillow 9.4.0	94.1	1	CVE-2023-4863 (8.8)	severity, EPSS
2	pyyaml 5.1	65.7	4	CVE-2019-20477 (9.8)	severity, depth
3	django 4.1.7	64.6	2	CVE-2025-64459 (9.1)	severity, depth
4	pyjwt 2.4.0	61.3	5	CVE-2026-102268 (9.1)	severity, depth
5	cryptography 39.0.1	55.2	6	CVE-2023-49083 (7.5)	severity, depth

Rank	Component	Score	CVSS-only rank	Driving vulnerability (CVSS)	Largest factors
6	idna 3.4	48.6	11	CVE-2024-3651 (7.5)	severity, exploit evidence
7	werkzeug 2.1.2	47.4	10	CVE-2024-34069 (7.5)	severity, depth
8	urllib3 1.26.9	46.6	9	CVE-2024-37891 (6.5)	severity, exploit evidence
9	certifi 2022.12.7	44.9	3	CVE-2023-37920 (9.8)	severity, age
10	sqlparse 0.3.1	43.7	7	CVE-2026-54284 (8.7)	severity, age

What changed relative to the baseline

The contextual ranking reorders 13 of the 14 components, and Kendall's τ between the two full rankings is 0.58, where 1.0 would mean identical order. The largest moves each have a clear cause in the factors:

- **Pillow is #1 in both rankings, but for different reasons.** The CVSS-only baseline puts it first because one of its recent flaws, CVE-2026-112393, now has a CVSS score of 9.8 (with an EPSS probability of only 0.01), and among the four components tied at 9.8 Pillow has the most vulnerabilities. The contextual ranking is driven instead by its bundled libwebp flaw, CVE-2023-4863, which has an EPSS probability of 1.00 and is in the CISA KEV catalog, so it is being exploited in the wild. Its score of 94.1 is far ahead of the next component (65.7).
- **Certifi falls from #3 to #9.** Its CVE-2023-37920 has a CVSS score of 9.8, but an EPSS probability of 0.001, no known exploit, and the package sits two levels deep, brought in by requests.
- **idna rises from #11 to #6** because NVD lists public exploit code for CVE-2024-3651.
- **PyYAML rises from #4 to #2** as a direct dependency whose CVE-2019-20477 scores 9.8.
- **sqlparse falls from #7 to #10 and oauthlib from #8 to #14.** Both are indirect dependencies with no exploitation evidence; oauthlib sits three levels deep.

Between October 23 and November 6 the vulnerability data itself changed: two new advisories appeared (CVE-2026-112398 in Pillow and CVE-2026-118669 in oauthlib), and NVD raised the CVSS score of CVE-2026-112393 from 9.2 to 9.8, which moved Pillow from #4 to #1 in the baseline. This is why every run records its analysis date and caches every response. Only idna enters the contextual top 10 (replacing oauthlib), because PyGoat has only 14 vulnerable components. The differentiation target in D1 therefore depends mainly on the larger test projects, which are evaluated in D3.

Explanations

Each ranked component carries one sentence naming its driving vulnerability, its two largest weighted contributions with their point values, its depth and path, the version's age and its baseline rank. Two examples from this run:

> #1 pillow@9.4.0 (score 94.1, baseline #1): driven by CVE-2023-4863 (CVSS 8.8); largest factors are CVSS severity 8.8 (+26.4) and a 1.00 EPSS exploit probability (+25.0); direct dependency; version 3.8 years old; 53 vulnerabilities; 2 data-quality flags.
>
> #9 certifi@2022.12.7 (score 44.9, baseline #3): driven by CVE-2023-37920 (CVSS 9.8); largest factors are CVSS severity 9.8 (+29.4) and the version's age (+7.8); 2 levels deep via requests@2.28.2 → certifi@2022.12.7; version 3.9 years old; 3 vulnerabilities; 1 data-quality flag.

Data-quality flags

Nine advisories have no CVE identifier (most are GitHub advisories about the OpenSSL libraries bundled in cryptography wheels), so EPSS and KEV do not apply to them and their exploit probability is zero. Every CVE has an EPSS score in this run, and one advisory carries no severity data at all and receives the 0.50 default. Each case is flagged in the output rather than silently filled in.

6. Testing

The project has 34 automated tests, run with pytest. They cover parsing, graph construction (including cycles and unreachable components), enrichment with recorded API responses (alias merging, withdrawn records, missing scores, offline replay), scoring, and the check that refuses a placeholder NVD API key. The scoring tests reproduce both worked examples from Section 4 of D1 exactly (94.9 and 34.2), and check the tie-breaking rules, that the driving vulnerability's factors are never mixed with another vulnerability's, every missing-data fallback, the explanation format, Kendall's τ, and the weight variation used in the sensitivity analysis.

7. Implementation Notes and Known Limitations

Three details that D1 leaves open were settled as follows. When two vulnerabilities in the same component have equal scores, the one with the higher severity, then the lower identifier, is the driving vulnerability. The baseline uses the same severity fallbacks as factor S, so an advisory without a CVSS score ranks by its qualitative rating in both lists. And an explanation reports the number of data-quality flags rather than listing each one; the full list is in the output files.

Two limitations observed during testing will be carried into the final report. OSV occasionally reports a vulnerability for a version outside the range stated in the NVD description; for example, CVE-2026-69249 is reported for cryptography 39.0.1 although NVD describes versions 42.0.0 through 48.0.0. The pipeline reports what OSV returns and does not second-guess it. Second, recently published CVEs are often scored only by the organization that published them and not yet by NIST; the pipeline then uses that organization's CVSS score as listed by NVD.

References

[1] OWASP Foundation, "CycloneDX Specification, Version 1.5," 2023. [Online]. Available: https://cyclonedx.org/docs/1.5/json/

[2] Google and OpenSSF, "Open Source Vulnerabilities (OSV) Database and API." [Online]. Available: https://osv.dev

[3] NIST, "National Vulnerability Database API v2.0." [Online]. Available: https://nvd.nist.gov/developers/vulnerabilities

[4] FIRST, "Exploit Prediction Scoring System (EPSS)." [Online]. Available: https://www.first.org/epss

[5] CISA, "Known Exploited Vulnerabilities Catalog." [Online]. Available: https://www.cisa.gov/known-exploited-vulnerabilities-catalog